

COMPSCI1XC3 Assignment 3

Hannah Fabik

July 31, 2024

1 Makefile

A makefile is provided to make compilation of the code easier. To use the makefile, simply type `make` into the command line to compile the program. This will generate an executable program file called `GA` which takes 5 inputs from the user at run time. It can then be run using

```
./GA <int POPULATION SIZE> <int MAX GENERATIONS> <float crossover rate>  
<float mutate rate> <float stop criteria>
```

To remove the executable files, type `make clean` into the terminal.

1.1 How it works

The makefile uses macros to make the file more editable and readable. Four of them are defined in this file:

- `CC`
- `CFLAGS`
- `EXE`
- `SRC`

The `CC` macro defines the compiler. In this case it is GNU's gcc.

The `CFLAGS` macro define the flags with which the files should be compiled. Here `-Wall` is used to enable all warnings from the compiler. While this is not necessary, it is helpful to show all the possible issues with the code. Next, the flag `-lm` is used to link the math library, `libm`, for the `math.h` header file. Finally, the flag `-g` is included but commented out to include debugging into the executable file. The executable can thus be used in either debugging or regular running mode when this is enabled.

The `EXE` macro defines the executable file, the target for the compilation. In this case it is `GA`. Lastly, the `SRC` macro defines the source files; the dependencies that are required to build the target. These include `OF.c` for the objective function, `functions.c` for the functions of the genetic algorithm, and `GA.c` for the main function.

Makefiles rely on rules made with the above components to specify how the files should actually

be compiled. The general format is:

```
target : dependencies
      rule
```

The target and dependencies are defined by the `EXE` and `SRC` macros as mentioned above. The rule is separated by a TAB space. It is of the form `<compiler> -o <target> <source> <flags>`. The `-o` flag represents the object file. The above defined macros are used to represent the items in the rule in this respective order, with the syntax `$(macro)`.

The final section of the makefile allows for the removal of the executable file. When the command `make clean` is called, the `GA` object is deleted using the command `rm`. The `-f` flag indicates the files to be removed, in this case just GA.

2 Results

The stopping criteria for every run in Tables 1-3 was set to $1e-7$. At first it was attempted with $1e-10$, but this took over 30 minutes to run with 10 variables so the stopping criteria was increased to speed up execution. Bounds for the Levy function were from $\{-5, 5\}$.

Table 1: `NUM_VARIABLES = 10`, Crossover Rate = 0.5, Mutation Rate = **0.05** for **Levy** function

Pop Size	Max Gen	Best Fitness	CPU time (Sec)
10	100	2.3649557481917118	0.001798
100	100	0.0002931514500049	0.009939
1000	100	0.0000352399253164	0.192472
10000	100	0.0000000000000000	0.859511
1000	1000	0.0000000615119046	0.762213
1000	10000	0.0000000891152409	2.824050
1000	100000	0.0000000850535025	4.008190
1000	1000000	0.0000000434940325	1.425422

Table 2: **NUM_VARIABLES = 10**, Crossover Rate = 0.5, Mutation Rate = **0.2** for **Levy** function

Pop Size	Max Gen	Best Solution	CPU time (Sec)
10	100	2.8254431354734435	0.001010
100	100	0.2053044790394943	0.010848
1000	100	0.0002686439057564	0.212916
10000	100	0.0000019330479227	12.352375
1000	1000	0.0000008172968886	1.889700
1000	10000	0.0000000754880881	13.960042
1000	100000	0.0000000996675514	13.699800
1000	1000000	0.0000000965105626	8.174846

Table 3: **NUM_VARIABLES = 50**, Crossover Rate = 0.5, Mutation Rate = **0.2** for **Levy** function

Pop Size	Max Gen	Best Solution	CPU time (Sec)
10	100	63.5678942889217709	0.003799
100	100	58.3439904005416068	0.061157
1000	100	49.3104955254965986	0.509441
10000	100	41.7873293419304588	15.097841
1000	1000	36.2648497546439188	4.872097
1000	10000	28.2076558059507860	49.664015
1000	100000	28.6700307740592173	496.318239
1000	1000000	—	DNF

The optimization algorithm was not able to generate a solution matching the $1e-7$ stopping criteria within 1 hour of running, thus it was terminated. From the above results, it can be seen that a lower mutation rate works better for the Levy function. Similarly, a large population also helps with the accuracy.

Table 4: `NUM_VARIABLES = 10`, Crossover Rate = CR, Mutation Rate = MR for **all** functions.

Function	Pop Size	Max Gen	CR	MR	Best Solution	CPU time (Sec)
Griewank	1000	100000	0.6	0.1	0.0148618164976486	148.444979
Levy	10000	100	0.5	0.05	0.0000000000000000	1.383942
Rastrigin	1000	10000	0.6	0.05	0.0000000000000000	0.031852
Schwefel	1000	100000	0.55	0.05	0.0001324367203779	143.856138
Trid*	1000	10000	0.6	0.3	0.0000672664020840	17.457382
Dixon-Price	1000	10000	0.6	0.05	0.6673551209260592	14.241601
Michalewicz*	1000	10000	0.7	0.3	-4.4065161023726995	18.964849
Powell	1000	1000	0.7	0.05	0.0000000000000000	0.003949
Styblinski-Tang**	1000	1000	0.5	0.05	-0.0000961154991046	0.409006

*The Trid function should have a minimum of -210 at $x = \{10, 18, 24, 28, 30, 30, 28, 24, 18, 10\}$, the Michalewicz a minimum of -9.66015, and Styblinski-Tang a minimum of -391.6599. However, this genetic algorithm determines the best fitness via $1/(\text{objective function})$, which has a minimum at 0 (including negative numbers). Thus, this algorithm is actually not seeking to minimize the function, but rather to bring it to 0. Any negative numbers that appear are as a result of a lucky starting population and the stopping criteria being set to a negative number.

**When evaluating the Styblinski-Tang function, 391.6599 was added to the objective function to centre the minimum at 0. Thus, the optimization algorithm was able to give accurate results for the x variables for this function.

3 Appendix

3.1 GA.c

```
// Include everything necessary here
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include "functions.h"
# ifndef M_PI
# define M_PI 3.14159265358979323846
# endif

void printArray(double *arr, int size)
{
    for(int i = 0; i < size-1; i++)
    {
        printf("%lf, ", arr[i]);
    }
    printf("%lf", arr[size-1]);
}

int main(int argc, char *argv[])
{
    // #####
    // #####
    // YOUR CODE: Handle the possible errors in input data given by
    // the user and say how to execute the code
    if (argc != 6)
    {
        printf("ERROR: correct format is 5 input numbers: <
            POPULATION SIZE> <MAX GENERATIONS> <crossover rate> <
            mutate rate> <stop criteria>\n");
        return 1;
    }
}
```

```

// YOUR CODE: Assign all inputs given by the user argv[i] like:
// POPULATION_SIZE, MAX_GENERATIONS, crossover_rate,
    mutate_rate, stop_criteria
int POPULATION_SIZE = atoi(argv[1]);
int MAX_GENERATIONS = atoi(argv[2]);
double crossover_rate = atof(argv[3]);
double mutate_rate = atof(argv[4]);
double stop_criteria = atof(argv[5]);

// #####
#####

/* YOUR CODE: You must change this part based on the lower and
upper bounds
1. based on what function is going to be minimized (let's say
    Levy)
2. given bound in https://www.sfu.ca/~ssurjano/optimization.html for each function.*/

// the number of variables (d)
int NUM_VARIABLES = 10;
// the lower bounds of variables (x_1, x_2, ..., x_d) where d=
    NUM_VARIABLES
double Lbound[] = {-5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5};
// the upper bounds of variable
double Ubound[] = {5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5};

/*For example: in Levy function x_i in [-10, 10], for all i =
    1, ..., d. This means:
lower bound = -10 for all x_i
upper bound = +10 for all x_i
if d =10 (or NUM_VARIABLES = 10) then:*/
//double Lbound[] = {-5.0, -5.0, -5.0, -5.0, -5.0, -5.0, -5.0,
    -5.0, -5.0, -5.0};
//double Ubound[] = {+5.0, +5.0, +5.0, +5.0, +5.0, +5.0, +5.0,
    +5.0, +5.0, +5.0};

/*Another example: in Griewank() if NUM_VARIABLES = 7, then:*/

```

```

//double Lbound[] = {-600.0, -600.0, -600.0, -600.0, -600.0,
    -600.0, -600.0, -600.0, -600.0, -600.0};
//double Ubound[] = {+600.0, +600.0, +600.0, +600.0, +600.0,
    +600.0, +600.0, 600.0, 600.0, 600.0};

// #####
// #####

// YOUR CODE: Here make all the initial print outs
printf("Genetic Algorithm is initiated.\n
    -----\n");
printf("The number of variables: %d\n", NUM_VARIABLES);
printf("Lower bounds: [");
printArray(Lbound, sizeof(Lbound)/sizeof(Lbound[0]));
printf("]\nUpper bounds: [");
printArray(Ubound, sizeof(Ubound)/sizeof(Ubound[0]));
printf("]\nPopulation Size: %d\n", POPULATION_SIZE);
printf("Max Generations: %d\n", MAX_GENERATIONS);
printf("Crossover Rate: %0.6lf\n", crossover_rate);
printf("Mutation Rate: %0.6lf\n", mutate_rate);
printf("Stopping criteria: %0.16lf\n\n", stop_criteria);

clock_t start_time, end_time;
double cpu_time_used;
start_time = clock();

// <YOUR CODE: Declare all the arrays you need here>
double population[POPULATION_SIZE][NUM_VARIABLES];
double new_population[POPULATION_SIZE][NUM_VARIABLES];
double fitness[POPULATION_SIZE];
double best_solution[NUM_VARIABLES]; /*best_solution = malloc(
    NUM_VARIABLES*sizeof(double));
double best_eval = 1e9;

// <YOUR CODE: Call generate_population function to initialize
    the "population"> like:
generate_population(POPULATION_SIZE, NUM_VARIABLES, population,
    Lbound, Ubound);

```

```

// iteration starts here. The loop continues until
// MAX_GENERATIONS is reached
// Or stopping criteria is met
for (int generation = 0; generation < MAX_GENERATIONS;
    generation++)
{
    // <YOUR CODE: Compute the fitness values using objective
    // function for
    // each row in "population" (each set of variables)> like:
    compute_objective_function(POPULATION_SIZE, NUM_VARIABLES,
        population, fitness);

    // <YOUR CODE: Here implement the logic of finding best
    // solution with minimum fitness value
    // and the stopping criteria>
    for (int i = 0; i < POPULATION_SIZE; i++)
    {
        if (fitness[i] < best_eval)
        {
            best_eval = fitness[i];
            memcpy(best_solution, population[i], sizeof(double)
                * NUM_VARIABLES);
            if (best_eval < stop_criteria)
            {
                goto end;
            }
        }
    }
}

// <YOUR CODE: Here call the crossover function>
crossover(POPULATION_SIZE, NUM_VARIABLES, fitness,
    new_population, population, crossover_rate);

// <YOUR CODE: Here call the mutation function>
mutate(POPULATION_SIZE, NUM_VARIABLES, new_population,
    population, Lbound, Ubound, mutate_rate);

```

```

        // Now you have the new population, and it goes to the
        // beginning of loop to re-compute all again
    }

end:
// #####
// #####
// You dont need to change anything here
// Here we print the CPU time taken for your code
end_time = clock();
cpu_time_used = ((double)(end_time - start_time)) /
    CLOCKS_PER_SEC;
printf("Results\n
    -----\n");
printf("CPU time: %f seconds\n", cpu_time_used);
// #####
// #####

// <Your CODE: Here print out the best solution and objective
// function value for the best solution like the format>
printf("Best solution found: (");
printArray(best_solution, sizeof(best_solution)/sizeof(
    best_solution[0])); //NUM_VARIABLES*sizeof(double));
printf(")\nBest fitness: %0.16lf\n", best_eval);

return 0;
}

```

3.2 functions.c

```

// Your CODE: Include everything necessary here
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <math.h>
#include "functions.h"

```

```
double generate_random(double min, double max)
{
    // Your CODE: implement a function to return a value between
    min and max
    //srand((unsigned int) time(NULL));
    double randomNum = ((float) rand()/(float) (RAND_MAX))*(max -
        min) + min;
    return randomNum;
}

int generate_int(int min, int max)
{
    // Your CODE: implement the function to return a random integer
    value
    int randInt = rand() % (max - min + 1) + min;
    return randInt;
}

// Function to initialize a random population
void generate_population(int POPULATION_SIZE, int NUM_VARIABLES,
    double population[POPULATION_SIZE][NUM_VARIABLES], double Lbound
    [NUM_VARIABLES], double Ubound[NUM_VARIABLES])
{
    // Your CODE: randomly initialize for all values in "population
    [i][j]"
    srand(time(NULL));
    for (int i = 0; i < POPULATION_SIZE; i++)
    {
        for(int j = 0; j < NUM_VARIABLES; j++)
        {
            population[i][j] = generate_int(Lbound[j], Ubound[j]);
        }
    }
}

// Function to compute the objective function for each member of
the population
```

```
void compute_objective_function(int POPULATION_SIZE, int
    NUM_VARIABLES, double population[POPULATION_SIZE][NUM_VARIABLES
], double fitness[POPULATION_SIZE])
{
    /* Your CODE: compute "fitness[i]" for each set of decision
        variables (individual) or each row in "population"
    by calling "Objective_function" */
    for(int i = 0; i < POPULATION_SIZE; i++)
    {
        fitness[i] = Objective_function(NUM_VARIABLES, population[i
        ]);
    }
}

void crossover(int POPULATION_SIZE, int NUM_VARIABLES, double
    fitness[POPULATION_SIZE], double new_population[POPULATION_SIZE
][NUM_VARIABLES], double population[POPULATION_SIZE][
NUM_VARIABLES], double crossover_rate)
{
    /* Your CODE: Implement the logic of crossover function here
        based on "fitness_probs" or each set
    of decision variables (individual) or each row in "population".
    And save the new population in "new_population"*/
    double total = 0;
    for (int i = 0; i < POPULATION_SIZE; i++)
    {
        total += 1/(1e-6+fitness[i]);
    }
    double fitness_probs[POPULATION_SIZE];
    double runningTotal = 0;
    for (int i = 0; i < POPULATION_SIZE; i++)
    {
        runningTotal += (1/(1e-6+fitness[i])) / total;
        fitness_probs[i] = runningTotal;
    }

    for(int i = 0; i < POPULATION_SIZE; i++)
    {
```

```
double r = generate_random(0, 1);
for(int j = 0; j < POPULATION_SIZE; j++)
{
    if (r < fitness_probs[j])
    {
        memcpy(new_population[i], population[j],
            NUM_VARIABLES * sizeof(double));
        break;
    }
}

int numParents = 0;
int parents[NUM_VARIABLES];
for (int k = 0; k < POPULATION_SIZE; k++)
{
    double r = generate_random(0, 1);
    if (r < crossover_rate)
    {
        parents[numParents] = k;
        numParents++;
    }
}

if (numParents == 0 || numParents == 1) {return;}

double temp[NUM_VARIABLES];
memcpy(temp, new_population[parents[0]], NUM_VARIABLES * sizeof
(double));

for(int i = 0; i < numParents-1; i++)
{
    int crossPos = generate_int(1, NUM_VARIABLES-1);
    int index = parents[i];
    int next = parents[i+1];
    for(int j = crossPos; j < NUM_VARIABLES; j++)
    {
        new_population[index][j] = new_population[next][j];
    }
}
```

```

    }
}
int index = numParents-1;
int crossPos = generate_int(1, NUM_VARIABLES-1);
for(int j = crossPos; j < NUM_VARIABLES; j++)
{
    new_population[parents[index]][j] = temp[j];
}
}

void mutate(int POPULATION_SIZE, int NUM_VARIABLES, double
    new_population[POPULATION_SIZE][NUM_VARIABLES], double
    population[POPULATION_SIZE][NUM_VARIABLES], double Lbound[
    NUM_VARIABLES], double Ubound[NUM_VARIABLES], double mutate_rate
)
{
    /*Your CODE: Implement the logic of mutation on "new_population
    " and then copy everything into "population"*/
    int total_gen = NUM_VARIABLES * POPULATION_SIZE;
    int num_muts = mutate_rate * total_gen;
    for (int i = 0; i < num_muts; i++)
    {
        int r = generate_int(0, total_gen-1);
        int row = r / NUM_VARIABLES;
        int col = r % NUM_VARIABLES;
        new_population[row][col] = generate_random(Lbound[col],
            Ubound[col]);
    }
    memcpy(population, new_population, sizeof(double) *
        POPULATION_SIZE * NUM_VARIABLES);
}

```

3.3 Makefile

```

CC = gcc
CFLAGS = -Wall -lm #-g
EXE = GA

```

```
SRC = functions.c OF.c GA.c

$(EXE): $(SRC)
    $(CC) -o $(EXE) $(SRC) $(CFLAGS)

clean:
    rm -f $(EXE)
```